

Encryptor

```
from cryptography.hazmat.primitives.ciphers import Cipher,
algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import import default_backend
import os

def encrypt_file(file_path, output_path, iv_output_path):
    # Generate a random 128-bit key
    key = os.urandom(16)
    # Generate a random initialization vector (IV)
    iv = os.urandom(16)

    # Print out the key and IV in hexadecimal format
    print(f"Encryption Key (hex): {key.hex()}")
    print(f"Initialization Vector (IV) (hex): {iv.hex()}")

    # Save the IV to a separate file
    with open(iv_output_path, 'wb') as iv_file:
        iv_file.write(iv)

    # Create a cipher object for AES-128 CBC mode
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv),
backend=default_backend())

    # Read the contents of the file to encrypt
    with open(file_path, 'rb') as file:
        plaintext = file.read()

    # Pad the plaintext to be compatible with block size
    padder = padding.PKCS7(128).padder()
    padded_plaintext = padder.update(plaintext) +
padder.finalize()
```

```
# Encrypt the padded plaintext
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded_plaintext) +
encryptor.finalize()

# Save the encrypted data to the output file without the
IV
with open(output_path, 'wb') as file:
    file.write(ciphertext)

if __name__ == '__main__':
    file_path = 'path/to/your/input/file' # Replace with
your file path
    output_path = 'path/to/your/encrypted/file' # Replace
with your encrypted file path
    iv_output_path = 'path/to/your/iv/file' # Replace with
your IV file path
    encrypt_file(file_path, output_path, iv_output_path)
```

```
openssl enc -aes-128-cbc -d -in encrypted_file.bin -out
decrypted_file.txt -K key_file_hex -iv iv_hex
```